DALHOUSIE UNIVERSITY

# Summarizing Data

## Location, variability, and outliers

**Prof. Gabriel Spadon, PhD**

Faculty of Computer Science

spadon@dal.ca

# 1 Lab 01 and your portfolio

Lab 01 is individual work, due September 17 before 16:05 Halifax time. The best 10 of 12 labs count for 15% of the course grade. Keep the repository and cloud notebooks private, with instructor access, and record the commit SHA and URL in `labs/lab-01/README.md`. After the deadline and the instructor's release, publish that commit and its history in your portfolio and present it at the next class meeting, if one follows. Do not rewrite or backdate assessment history; corrections use new commits. Generative AI output is prohibited on labs.

# 2 Why summarize data, and what kind of variable is it?

## 2.1 Why summarize

**What it is for** Data are often too large to understand by reading them. Reading every cell of the Heart Disease table, 918 rows by 12 columns, is impractical. A summary gives an overview of key properties of the *distribution*, meaning how the values are spread across the observations. Plots and source rows keep the context that a handful of numbers can hide.

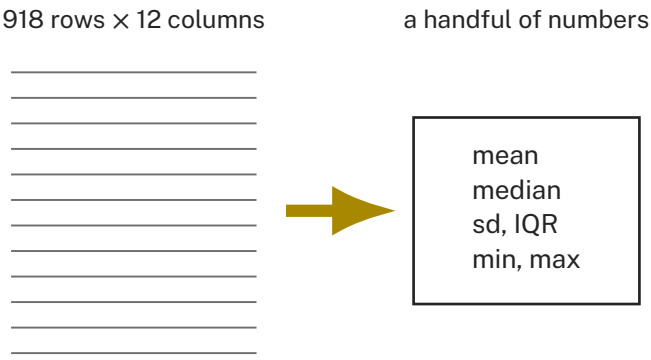



**Figure 1.** Rows condensed into a summary. Summaries condense the table; they do not replace checking its distributions and source observations.

## 2.2 Types of variables

**What it is for** The type of a variable decides which summaries are defined for it.

**The types** A numeric variable is *discrete* (a count, such as the number of visits) or *continuous* (a measurement, such as temperature). A categorical variable is *nominal* (unordered labels, such as blood group) or *ordinal* (ordered labels, such as low, moderate, high). Table 1 gives one example of each.

A nominal variable has no meaningful mean, even when its categories are stored as integers and the software returns a number. Storage type does not decide analytical role. R stores the whole-number values below as doubles, and pandas stores them as `int64`. Either way they can represent a discrete variable.

**R**

```r
x <- c(2, 4, 4, 5, 7, 9, 12, 41)
class(x)   # "numeric"
str(x)
# num [1:8] 2 4 4 5 7 9 12 41
```

**Python**

```python
import pandas as pd
x = pd.Series(
    [2, 4, 4, 5, 7, 9, 12, 41])
print(x.dtype)   # int64
```

**Table 1.** Variable types and what their values represent

| Type | Example | Meaning of the values |
|---|---|---|
| Numeric, discrete | number of clinic visits | counts taking separate possible values |
| Numeric, continuous | temperature in degrees Celsius | measurements on a continuous scale, recorded to finite precision |
| Nominal categorical | recorded blood group | labels without a numerical order |
| Ordinal categorical | low, moderate, high severity | ordered labels; the gaps between categories need not be equal |

# 3 Three questions to ask of a variable

**The three questions**

**Q1.** What is the "typical or common value" of a variable?
**Q2.** What is the "variability" in the values of a variable?
**Q3.** Are there unusual or unexpected values in the data?

These three questions organize the rest of the class. Each one has a small family of statistics attached to it. The same eight practice values are used throughout, $x = 2, 4, 4, 5, 7, 9, 12, 41$. In R, `c(...)` collects values into a vector. In Python, `pd.Series([...])` creates a labelled column.

# 4 Q1. Statistics of location

## 4.1 Mean, median, mode

**What it is for** A statistic of location gives the typical or common value of a variable.

**Definitions** The sample mean is

$$\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i.$$

The **median** has at least half the observations at or below it and at least half at or above it, allowing ties. For numeric data, sort and pick the middle value; if there are an even number of values, calculate the mean of the two in the middle. The **mode** is the most frequently occurring value, and it is the only one of these three summaries that applies to unordered nominal categories.

**Worked example**

The eight values are $x = 2, 4, 4, 5, 7, 9, 12, 41$, so $n = 8$.

- The mean is $\bar{x} = (2 + 4 + 4 + 5 + 7 + 9 + 12 + 41)/8 = 84/8 = 10.5$.
- The two middle values of the sorted vector are $5$ and $7$, so the median is $(5 + 7)/2 = 6$.
- The value $4$ occurs twice and every other value once, so the mode is $4$.

The single value $41$ pushes the mean to $10.5$ while the median stays at $6$.

**R**

```r
mean(x)     # 10.5
median(x)   # 6

# ignore missing values
mean(c(2, NA, 4), na.rm = TRUE) # 3

# mode() reports storage, not frequency
tab <- table(x)
names(tab)[tab == max(tab)] # "4"
```

**Python**

```python
print(x.mean())     # 10.5
print(x.median())   # 6.0

# pandas skips NaN by default
y = pd.Series([2, None, 4])
print(y.mean())     # 3.0

x.mode().tolist()   # [4]
```

**Differences to know**

A missing value is recorded as `NA` in R and as `NaN` in a numeric pandas column. R requires `na.rm = TRUE` to omit it, and pandas omits it by default. Both examples average the two observed values, 2 and 4, and return 3. Report the observed count and the missing count separately. R's `mode()` reports the storage mode, not the most frequent value, so the listing counts values with `table`. The R frequency table and pandas' `mode` both return all tied modes. A notebook cell displays only the value of its last line, so the Python listings wrap earlier results in `print`.

## 4.2 Getting an accurate "picture" of your data

**What it is for** The mean and the median can describe the same data very differently. This subsection shows when and why.

> *Ten people have an average income of $50,000. Replace the highest earner with someone earning far more. The new average can be arbitrarily large, while the two middle incomes stay exactly the same.*

The median does not change. Replacing the largest of ten values with a larger value leaves the fifth and sixth values unchanged. The mean has no such protection. It is a sum divided by $n$, so one extreme value moves it without limit. The median is a *robust* statistic and the mean is a *non-robust* one.

Figure 2 shows the same effect in a histogram of the square footage of 25 homes. Twenty-three homes lie between 1 400 and 2 800 square feet and two are near 7 000.

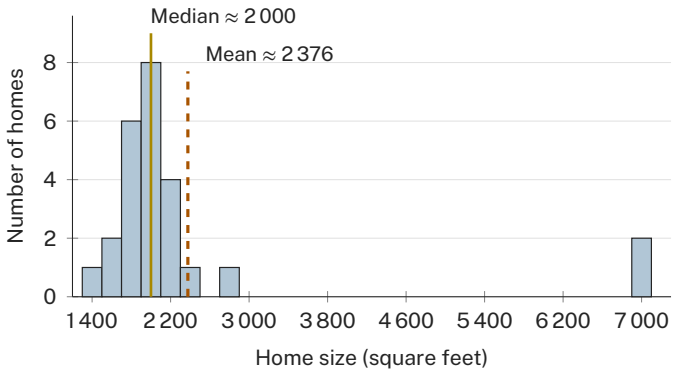



**Figure 2.** Histogram of home square footage with mean and median, reconstructed after Statology [1]

The bar heights are reconstructed from the original Statology illustration: 1 400 : 1, 1 600 : 2, 1 800 : 6, 2 000 : 8, 2 200 : 4, 2 400 : 1, 2 800 : 1 and 7 000 : 2, for $n = 25$. Using bin midpoints, the median falls

in the 2 000 bin and the mean is $59\,400/25 = 2\,376$. The two distant homes pull the mean above the median. These are midpoint estimates from the displayed bins, not exact statistics recovered from unavailable individual home sizes.

### 4.3 Skewness and the order of the three statistics

**What it is for** The order of the mean, median and mode tells you which way a distribution leans.

**The rule** A skewed distribution has a longer tail on one side. Figure 3 shows how the order of the three statistics shifts with the skew.

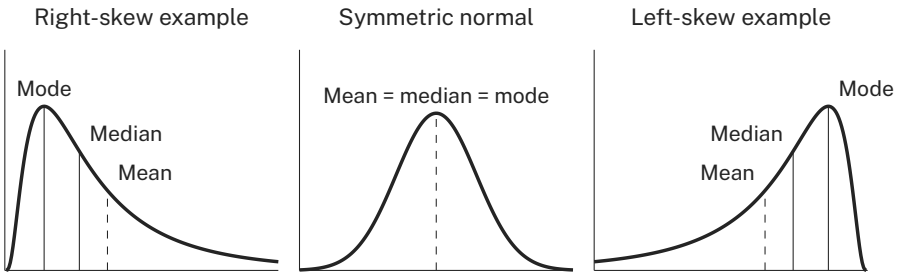



**Figure 3.** Mean, median and mode order under skew

Under right skew, mode < median < mean. The left-skew example reverses the order. This ordering is common but does not hold for every skewed distribution; von Hippel shows how discrete and multimodal data break it [2]. The symmetric normal example has equal mean, median and mode. In the running example the mode is 4, the median 6 and the mean 10.5, the right-skew order produced by the single value 41.

**Reporting location under skew**

Report the mean and the median together whenever the distribution may be skewed. A large gap between them is itself a finding. The distribution may have a long tail or contain values generated by a different mechanism, so check the plot and the measurement process. Choose the summary according to the question. The gap alone does not select the right statistic.

## 5 Q2. Statistics of variability, spread, or dispersion

### 5.1 Variance and standard deviation

**What it is for** A statistic of variability says how far the values lie from one another. Variance and standard deviation measure spread around the mean.

**Formulas**

$$s_x^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2, \qquad s_x = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2}.$$

The divisor $n-1$ makes these the *sample* variance $s_x^2$ and sample standard deviation $s_x$. Use the sample convention in Lab 01. Dividing the squared deviations by $n$ instead describes variation across the complete set of observed values. The two conventions give different numbers, so name the divisor when reporting either statistic.

**Units** Variance is in the *squared* units of the variable. If $x$ is in dollars, $s_x^2$ is in dollars squared, which is hard to interpret. The standard deviation is the square root, so it returns to the units of $x$ and can be read beside the mean. That is why the standard deviation is the one usually reported.

**Worked example**

For $x = 2, 4, 4, 5, 7, 9, 12, 41$ with $\bar{x} = 10.5$, the squared deviations are $72.25$, $42.25$, $42.25$, $30.25$, $12.25$, $2.25$, $2.25$ and $930.25$, summing to $1134$. Therefore

$$s_x^2 = \frac{1134}{7} = 162, \qquad s_x = \sqrt{162} = 12.73.$$

The single observation $41$ contributes $930.25$ of the $1134$ total, which is 82% of the variability from one of eight observations.

**R**

```r
var(x)   # 162      divisor n - 1
sd(x)    # 12.72792

# Descriptive variance with divisor n
var(x) * (length(x) - 1) / length(x)
# 141.75
```

**Python**

```python
print(x.var())        # 162.0  ddof = 1
print(x.std())        # 12.727922...

# Descriptive variance with divisor n
print(x.var(ddof=0))  # 141.75
x.std(ddof=1)         # 12.727922...
```

**Differences to know**

R's `var` and `sd`, and pandas' defaults, divide by $n - 1$. In pandas, `ddof=1` subtracts one from the observed count in the divisor, and `ddof=0` uses $n$. The sample standard deviation is $12.73$, while the divisor-$n$ version is $\sqrt{141.75} = 11.91$.

## 5.2 Percentiles, quantiles, quartiles

**What it is for** Quantiles mark chosen cumulative proportions of the data. They give robust measures of spread and the fences of the boxplot rule.

**Definitions** The $p$th percentile locates the lower $p\%$ of a distribution. In samples, ties and interpolation mean that exactly $p\%$ need not lie below the reported value. Dividing the data into four parts gives the *quartiles*, the 25th, 50th and 75th percentiles, written $Q_1$, $Q_2$ and $Q_3$. The second quartile is the median. Figure 4 shows the four equal regions on a continuous distribution.

## 5.3 Sample-quantile definitions

**What it is for** Software offers several sample-quantile definitions, and they give different numbers on small samples. You need to know which one you used. Hyndman and Fan catalogue nine of them, numbered as in R [3].

**The rule** For Lab 01, use R's type-7 definition, which matches pandas' linear interpolation. Sort the $n$ observed values, then compute the position $h = 1 + (n - 1)p$, counting positions from 1. If $h$ lies between two positions, interpolate linearly between their values.

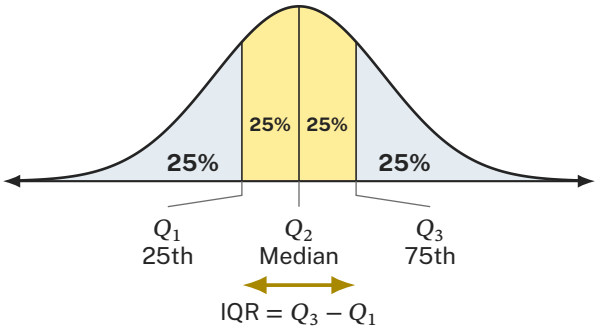



**Figure 4.** Quartiles and the interquartile range on a normal curve

The cut points enclose four equal probability regions. Unlike a finite sample, a normal distribution has no finite minimum or maximum. Sample 0th and 100th percentiles are the observed minimum and maximum, not the ends of this curve.

**Worked example**

The sorted values are $2, 4, 4, 5, 7, 9, 12, 41$, with $n = 8$.

- For $Q_1$, $p = 0.25$ gives $h = 1 + 7(0.25) = 2.75$. Positions 2 and 3 both hold 4, so $Q_1 = 4$.
- For $Q_3$, $p = 0.75$ gives $h = 1 + 7(0.75) = 6.25$, one quarter of the way from position 6 (value 9) to position 7 (value 12), so $Q_3 = 9 + 0.25(12 - 9) = 9.75$.

**R**

```r
quantile(x, c(0.25, 0.75), type = 7)
#  25%  75%
# 4.00 9.75          default type 7

fivenum(x)       # hinges, not type 7
# 2.0 4.0 6.0 10.5 41.0
```

**Python**

```python
x.quantile([0.25, 0.75],
           interpolation="linear")
# 0.25  4.00
# 0.75  9.75   linear == R type 7
```

**Differences to know**

R's `fivenum` and default `boxplot` use Tukey's hinges [4], a different rule for dividing a sorted sample. The upper hinge is the median of the upper half $7, 9, 12, 41$, which is 10.5, while the type-7 upper quartile is 9.75. State which rule you use. The screening code in the CSV walkthrough uses type 7 in both languages.

## 5.4 Interquartile range and range

**What it is for** Both measure spread with a single difference. The IQR measures the middle half, and the range measures the whole sample.

**Formulas**

$$\text{IQR} = Q_3 - Q_1, \qquad \text{range} = x_{(n)} - x_{(1)},$$

where $x_{(1)}$ and $x_{(n)}$ are the smallest and largest observed values. Table 2 gives both for the running example.

**Robustness** The IQR describes the middle half of the data and ignores the tails, so it is robust. Moving the largest observation from 41 to 4,100 leaves it at 5.75. The range uses only the two most extreme observations, so it is determined entirely by them, and under repeated sampling it tends to grow with

**Table 2.** Interquartile range and range for the running example

| Statistic | Definition | Value for the running example |
|---|---|---|
| Interquartile range | $\text{IQR} = Q_3 - Q_1$ | $9.75 - 4 = 5.75$ |
| Range | maximum minus minimum | $41 - 2 = 39$ |

sample size even when the underlying distribution does not change. In the running example the range is $41 - 2 = 39$ and the IQR is $9.75 - 4 = 5.75$, nearly a factor of seven apart.

**R**

```r
IQR(x)          # 5.75
range(x)        # 2 41
diff(range(x))  # 39
```

**Python**

```python
q1, q3 = x.quantile([0.25, 0.75],
                    interpolation="linear")
print(q3 - q1)           # 5.75
print(x.min(), x.max())  # 2 41
x.max() - x.min()        # 39
```

**Differences to know**

R's `range()` returns the two endpoints, not their difference, so `diff(range(x))` gives the spread. pandas has no `IQR` method, so the subtraction is written out, which also makes the quantile definition visible.

# 6 Q3. Outliers and unknown values

## 6.1 What an outlier is

**What it is for** Unusual values can be errors or the most important observations in a study. Either way they must be found before they distort the other summaries.

**Definition and rule** An outlier is an observation unusual enough, relative to the others, to prompt investigation of its origin or measurement process. A frequently used screening criterion is Tukey's **boxplot rule** [4]. It flags observations outside the interval

$$\left[\, Q_1 - 1.5 \times \text{IQR},\;\; Q_3 + 1.5 \times \text{IQR} \,\right].$$

**Worked example**

For $x = 2, 4, 4, 5, 7, 9, 12, 41$ we have $Q_1 = 4$, $Q_3 = 9.75$ and $\text{IQR} = 5.75$, so $1.5 \times \text{IQR} = 8.625$ and the interval is

$$[\, 4 - 8.625,\;\; 9.75 + 8.625 \,] = [-4.625,\; 18.375].$$

Only $41$ lies outside it, so the rule flags exactly one value. The type-7 diagram below places whiskers at the most extreme observations inside these fences, $2$ and $12$. R's default boxplot uses hinges, giving $Q_3 = 10.5$ and fences $-5.75$ and $20.25$ for this sample. Both approaches flag $41$, but their boxes and numerical fences differ.

Figure 6 labels the parts of a boxplot on a second scale. Dashed lines are the fences, and each whisker stops at the last observation inside them.

Here $Q_1 = 7.72$, $Q_3 = 8.42$, and $\text{IQR} = 0.70$, so the fences are $6.67$ and $9.47$. Every separate circle lies outside those fences. The whiskers, at $6.82$ and $9.10$, stop inside them.

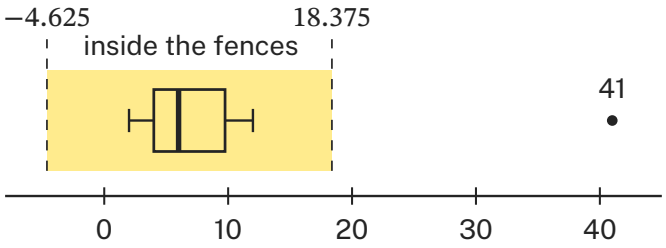



**Figure 5.** The running example, with type-7 quartiles, the two fences, observed whisker endpoints, and the single flagged value 41.

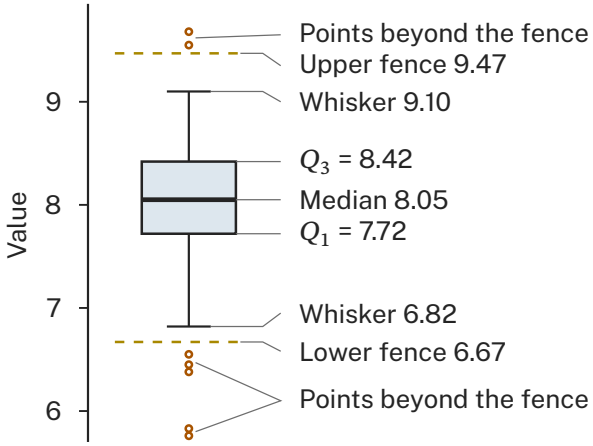



**Figure 6.** Anatomy of a boxplot with labelled parts

## 6.2 Missing and obviously wrong values

**What it is for** Two further kinds of unusual value are *missing or unknown* values and *obviously wrong* values, for example values of the wrong type. Both must be located and counted before any summary is trusted.

**Worked example** Add one missing reading to the running example. The boxplot rule still flags only 41, and the missing value sits at position 9 in R, which counts from 1, and at index 8 in pandas, which counts from 0.

**R**

```r
x <- c(2, 4, 4, 5, 7, 9, 12, 41, NA)
boxplot.stats(x)$out
# 41
which(is.na(x))
# 9
sum(is.na(x))
# 1
```

**Python**

```python
x = pd.Series(
    [2, 4, 4, 5, 7, 9, 12, 41, None])
q1, q3 = x.quantile([0.25, 0.75],
                    interpolation="linear")
iqr = q3 - q1
print(x[(x < q1 - 1.5 * iqr) |
        (x > q3 + 1.5 * iqr)])
# 7    41.0
x.index[x.isna()].tolist()
# [8]
```

**Differences to know**

`boxplot.stats` builds its fences from Tukey's hinges rather than from `quantile` type 7, so the two approaches can disagree at the margin. Here both flag only 41.

**Before changing a flagged value**

A flagged value is a question, not a verdict. Before changing or deleting a value, check the measurement process. An outlier can be a data-entry error, a unit mix-up, a legitimate rare event, or the single most interesting observation in the study. Record what you did and why, and report results with and without the flagged values when the conclusion changes.

## 7 Global summaries

**What it is for** One call returns several of the summaries above at once, so a whole variable can be described in a single step.

**The functions** R's `summary` and pandas' `describe` collect the location and quartile summaries. Frequency tables, `table` in R and `value_counts` in pandas, count observations in each category and are the summary for a categorical variable. Every number printed below matches a hand calculation from the earlier sections.

**R**

```r
x <- c(2, 4, 4, 5, 7, 9, 12, 41)
summary(x)
#  Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#  2.00    4.00    6.00   10.50    9.75   41.00
table(c("a", "b", "a", "c"))
# a b c
# 2 1 1
```

**Python**

```python
x = pd.Series(
    [2, 4, 4, 5, 7, 9, 12, 41])
print(x.describe().round(2))

s = pd.Series(list("abac"))
print(s.value_counts())

print(x.agg(["mean", "median", "std"]))
```

**Printed**

```
count     8.00
mean     10.50
std      12.73
min       2.00
25%       4.00
50%       6.00
75%       9.75
max      41.00
```

```
a    2
b    1
c    1

mean      10.500000
median     6.000000
std       12.727922
```

**Differences to know**

R's `summary()` prints the mean but not the standard deviation. pandas' `describe()` prints both, and its `std` uses `ddof=1`, which matches R's `sd()`. The two outputs look different while describing the same numbers. `agg` picks out named summaries only.

## 8 Common misconceptions

- The mean is not always the right summary. For a skewed variable, the median can describe the middle case more usefully. The mean remains relevant to questions about totals or expected values. Choose the summary to match the question.
- The mode is not the peak of a histogram. The mode is the most frequent *value*; a histogram's peak is the most frequent *bin*, which depends on the bin width you chose.

- An outlier is not automatically an error. The boxplot rule is a screening device with no knowledge of the measurement process, so a valid extreme measurement can be flagged. Check its units and source record before changing it.
- Quartiles are not unambiguous. Type 7 and R's hinges are two conventions, and they give 9.75 and 10.5 for the same upper quartile. The differences matter in small samples and near outlier thresholds. State the definition rather than assuming agreement.
- Standard deviation and variance carry the same information; only the units differ. Report the standard deviation because it is interpretable next to the mean.

## 9 Summaries in R or Python on a small table

This worked example applies the statistics of this class, in R and in Python, to a small fixed CSV whose `temperature_c` column holds the eight running-example values. Both scripts write their results to CSV files in `results/`, and the `temperature_c` row of `results/summaries.csv` reproduces the hand calculations of the earlier sections.

## 10 From a CSV to an analysis

In the following small dataset, each row represents one sensor reading. The three measurements are temperature in degrees Celsius, oxygen saturation in percent, and depth in metres. `record_id` is a stable row identifier and `site` defines two comparison groups. Neither is a measurement to average. This nine-row example cannot satisfy Lab 01's requirement for at least 100 observations from an approved source.

Create a folder for this practice and save the following text as `practice.csv`. CSV means comma-separated values. The first line gives column names; the blank temperature field on the last line records a missing measurement. A missing reading is different from a measured zero.

```
record_id,site,temperature_c,oxygen_percent,depth_m
r01,A,2,70,1
r02,A,4,80,2
r03,A,4,80,2
r04,A,5,85,3
r05,B,7,90,3
r06,B,9,95,4
r07,B,12,105,4
r08,B,41,130,10
r09,A,,75,2
```

**Listing 1.** The complete practice dataset.

### 10.1 Read the table and summarize its measurements

Choose one language. Place `practice.R` or `practice.py` beside the CSV, then assemble the following code blocks in order. Run R with `Rscript practice.R` or Python with `python practice.py` from that folder. In a cloud notebook, upload the CSV to the working folder and run the cells in order. Python needs pandas and Matplotlib; the R example uses functions included with R.

Each construct in the first block does one job. `for` visits the names in `numeric_cols` one at a time. `observations[[column]]` in R and `observations[column]` in Python select that column by name and

leave the table unchanged. `x[!is.na(x)]` and `x.dropna()` keep the non-missing values, here called `observed`. R starts an empty table with `data.frame()` and adds one row per column with `rbind`. Python collects one dictionary per column in the list `rows` and turns the list into a table with `pd.DataFrame`. `dir.create` and `Path("results").mkdir` create the folder the summary CSV is written to. The practice columns all have at least two observed values. In a real extract, check this before computing a sample standard deviation.

```r
# Run from the folder containing practice.csv.
observations <- read.csv("practice.csv", stringsAsFactors = FALSE)
numeric_cols <- c("temperature_c", "oxygen_percent", "depth_m")
dir.create("results", showWarnings = FALSE)

summaries <- data.frame()
for (column in numeric_cols) {
  x <- observations[[column]]
  observed <- x[!is.na(x)]
  quartiles <- quantile(observed, c(0.25, 0.75), type = 7)
  row <- data.frame(
    variable = column, n = length(observed), missing = sum(is.na(x)),
    mean = mean(observed), median = median(observed),
    variance = var(observed), sd = sd(observed),
    q1 = unname(quartiles[1]), q3 = unname(quartiles[2]),
    iqr = unname(quartiles[2] - quartiles[1]),
    range = max(observed) - min(observed)
  )
  summaries <- rbind(summaries, row)
}
write.csv(summaries, "results/summaries.csv", row.names = FALSE)
```

**Listing 2.** R import and summaries.

```python
# Run from the folder containing practice.csv.
from pathlib import Path

import matplotlib.pyplot as plt
import pandas as pd

observations = pd.read_csv("practice.csv")
numeric_cols = ["temperature_c", "oxygen_percent", "depth_m"]
Path("results").mkdir(exist_ok=True)

rows = []
for column in numeric_cols:
    x = observations[column]
    observed = x.dropna()
    q1, q3 = observed.quantile([0.25, 0.75], interpolation="linear")
    rows.append({
        "variable": column, "n": observed.size, "missing": x.isna().sum(),
        "mean": observed.mean(), "median": observed.median(),
        "variance": observed.var(ddof=1), "sd": observed.std(ddof=1),
        "q1": q1, "q3": q3, "iqr": q3 - q1,
        "range": observed.max() - observed.min(),
    })
pd.DataFrame(rows).to_csv("results/summaries.csv", index=False)
```

**Listing 3.** Python import and summaries.

For temperature, the output must show 8 observed values and 1 missing value, mean 10.5, median 6, sample variance 162, sample standard deviation about 12.73, $Q_1 = 4$, $Q_3 = 9.75$, IQR 5.75, and range 39. Check these against the hand calculations before adapting the code to an environmental extract.

## 10.2 Make one comparison

Define the groups from the question before screening unusual values. The next loop selects site A and site B in turn, then calculates their observed counts, missing counts, means, and medians. In R, the condition inside brackets selects rows. In pandas, `loc[condition, column]` selects rows and a column. Site A has 4 observed temperatures, 1 missing temperature, and mean 3.75; site B has 4 observed temperatures, no missing values, and mean 17.25. The value 41 contributes to that difference.

```r
groups <- data.frame()
for (site in c("A", "B")) {
  x <- observations$temperature_c[observations$site == site]
  row <- data.frame(
    site = site, n = sum(!is.na(x)), missing = sum(is.na(x)),
    mean = mean(x, na.rm = TRUE), median = median(x, na.rm = TRUE)
  )
  groups <- rbind(groups, row)
}
write.csv(groups, "results/groups.csv", row.names = FALSE)
```

**Listing 4.** R comparison.

```python
rows = []
for site in ["A", "B"]:
    x = observations.loc[observations["site"] == site, "temperature_c"]
    rows.append({
        "site": site, "n": x.count(), "missing": x.isna().sum(),
        "mean": x.mean(), "median": x.median(),
    })
pd.DataFrame(rows).to_csv("results/groups.csv", index=False)
```

**Listing 5.** Python comparison.

## 10.3 Trace a flag back to its row

This example screens all observed temperatures together, using type-7 quartiles. The condition marks values below the lower fence or above the upper fence. The saved row retains its identifier and site, so you can trace the flag back to the input. Only `r08`, the reading of 41, is flagged. Screening within each site would ask a different question and can give a different result. For Lab 01, state which population of rows your fences describe and justify that choice.

```r
x <- observations$temperature_c
quartiles <- quantile(x, c(0.25, 0.75), na.rm = TRUE, type = 7)
iqr <- quartiles[2] - quartiles[1]
flag <- !is.na(x) & (x < quartiles[1] - 1.5 * iqr |
                     x > quartiles[2] + 1.5 * iqr)
write.csv(observations[flag, ], "results/flagged.csv", row.names = FALSE)
```

**Listing 6.** R screening with type-7 quartiles.

```
x = observations["temperature_c"]
q1, q3 = x.quantile([0.25, 0.75], interpolation="linear")
iqr = q3 - q1
flag = x.notna() & ((x < q1 - 1.5 * iqr) | (x > q3 + 1.5 * iqr))
observations.loc[flag].to_csv("results/flagged.csv", index=False)
```

**Listing 7.** Python screening with linear interpolation.

For a real measurement, a statistical flag starts an investigation. Read the publisher's variable description and quality flags, check the units, and locate the original record. Retain a valid unusual value; correct or exclude a value only with documented evidence. This table has no publisher quality record, so it gives no evidence that 41 is a sensor error. In Lab 01, report the source check and your decision. If no values are flagged, inspect and describe the most extreme observations instead.

## 10.4 Save plots and document the run

The histogram uses the same ten-degree bins in both languages. It displays the eight observed temperatures and omits the missing reading. The scatter plot shows temperature against oxygen saturation for rows where both are observed. As in Class A, overlapping points can hide observations. Label every axis with the variable and its unit, and give the plotted count in your report. Association in a scatter plot does not establish cause and effect.

R's `pdf` opens a plot file and `dev.off` closes it. Matplotlib creates a figure and axes, saves the figure with `savefig`, and closes it with `close`. Each script also writes an environment record. Run the whole script again from a fresh session and confirm that it recreates the three tables, two plots, and environment file without manual edits.

For Lab 01, keep the raw download unchanged and record its publisher, URL, licence, access date, filter choices, units, and missing-value codes in the README. A simple report can state the question, describe the extract, present one summary table and two plots, discuss checked unusual values, and state a limitation. A standard word processor exported to PDF is sufficient this week; automated reporting tools are introduced later.

```
pdf("results/temperature-histogram.pdf", width = 6, height = 4)
hist(x, breaks = c(0, 10, 20, 30, 40, 50), right = FALSE,
     main = "Practice temperature readings", xlab = "Temperature (C)",
     ylab = "Number of readings", col = "grey80")
invisible(dev.off())

pdf("results/temperature-oxygen.pdf", width = 6, height = 4)
plot(observations$temperature_c, observations$oxygen_percent,
     xlab = "Temperature (C)", ylab = "Oxygen saturation (%)", pch = 16)
invisible(dev.off())
writeLines(capture.output(sessionInfo()), "results/environment.txt")
```

**Listing 8.** R plots and environment record.

```
fig, ax = plt.subplots(figsize=(6, 4))
ax.hist(x.dropna(), bins=[0, 10, 20, 30, 40, 50], color="grey", edgecolor="black")
ax.set(title="Practice temperature readings", xlabel="Temperature (C)",
       ylabel="Number of readings")
fig.savefig("results/temperature-histogram.pdf", bbox_inches="tight")
plt.close(fig)

fig, ax = plt.subplots(figsize=(6, 4))
ax.scatter(observations["temperature_c"], observations["oxygen_percent"])
ax.set(xlabel="Temperature (C)", ylabel="Oxygen saturation (%)")
fig.savefig("results/temperature-oxygen.pdf", bbox_inches="tight")
plt.close(fig)
pd.show_versions(as_json="results/environment.json")
```

**Listing 9.** Python plots and environment record.

## 11 Record and submit your work

A repository stores files and their revision history. A commit records a snapshot; its full SHA identifies that snapshot. A push sends local commits to GitHub. For Week 1, create a new **private** repository, give the instructor access in its collaborator settings, and keep it private until sharing is opened. Store Lab 01 under `labs/lab-01/`.

You can use GitHub's browser interface to create or upload files and choose *Commit changes*. Commit a meaningful stage of work each time, such as the imported table audit, the analysis, or the report. Before uploading, check that files contain only material allowed in the assessed repository. Keep the original download separate and document how to retrieve it.

If using local Git, first clone your private repository using the URL shown under its *Code* button. Open a terminal in that local repository. After editing the lab files, use the following sequence.

```
git remote -v
git status
git add labs/lab-01
git commit -m "Record Lab 01 analysis and evidence"
git push
git rev-parse HEAD
```

`git remote -v` shows the destination; confirm it is your private repository. `status` lists changed files, `add` selects the lab files, and `commit` records them. After a successful push, open that commit on GitHub and check its files. Copy the full SHA from the commit page or `git rev-parse HEAD`. Record it and the commit URL in that item's `README.md`, then commit and push that record by the deadline; the marker reads the README on the default branch and assesses the snapshot it names. A branch link does not identify one fixed version.

Keep corrections as new commits. Do not amend, rebase, squash, delete, replace, force-push, or backdate assessment history. After the deadline and the instructor's release, publish the submitted commit with its history and present that version at the next class meeting, if one follows. Keep other unreleased work and private feedback out of the entire history being published. Follow syllabus A.6.1 if a private portfolio alternative is required.

**Repository setup** docs.github.com/en/repositories/creating-and-managing-repositories/quickstart-for-repositories
**Committing in the browser** docs.github.com/en/repositories/working-with-files/managing-files/editing-files

## 12 Summary

Summarize data by calculating statistics of **location** (mean, median and mode) and statistics of **variability** (variance, standard deviation, interquartile range and range). Only the mode applies to nominal categories. Flag possible outliers using the boxplot rule, then investigate the source and context of values outside $[\, Q_1 - 1.5 \times \text{IQR},\ Q_3 + 1.5 \times \text{IQR}\,]$.

Different statistics provide a different "picture" of the data and may be distorted by skewness of the distribution and by the presence of outliers. Choose summaries for the question, and check plots, missingness, units and software defaults. Report the choice of summary and its interpretation alongside the number.

The variable's type decides which summaries exist at all. The software's defaults decide which version of a summary you actually computed. Between R and Python the substantive differences to remember are the treatment of missing values, the `ddof` divisor for variance and standard deviation, and the sample-quantile definition behind any quartile.

## References

[1] Z. Bobbitt. *How Do Outliers Affect the Mean?* Statology. Jan. 29, 2020. URL: `https://www.statology.org/how-do-outliers-affect-the-mean/`.

[2] P. T. von Hippel. "Mean, median, and skew: correcting a textbook rule". In: *Journal of Statistics Education* 13.2 (2005).

[3] R. J. Hyndman and Y. Fan. "Sample quantiles in statistical packages". In: *The American Statistician* 50.4 (1996), pp. 361–365.

[4] J. W. Tukey. *Exploratory Data Analysis*. Reading, MA: Addison-Wesley, 1977.